Variables, Assignment, and Program State

Lecture 5

CS 1001

Kirk Duran

Today

- How a program gives a value a name and uses that value later.
- Why assignment evaluates the right-hand side before changing program state.
- How Python connects names, namespaces, objects, and object identities.
- How reassignment, dynamic typing, and immutable strings fit this model.
- How comments, annotations, and good variable names make state easier to understand.

Key idea

This is the first lecture in which the order of multiple statements changes what later statements mean.

Part 1: From Expressions to Program State

Expressions Produce Values; Programs Can Retain Them

Lecture 3

`10 + 5` evaluates to one value, `15`. Nothing in the expression gives that result a name for later use.

Lecture 5

`score = 10 + 5` evaluates the same expression and then associates its result with the name `score`.

Image Placeholder

`expression-versus-stateful-program.png`

Key idea

A named result can influence statements that execute later.

A Variable Is a Name Associated with a Value

Definition

A **variable** is a name that is currently associated with a value.

Statement

```
score = 10
```

Association after execution

`score` $\longrightarrow$ `10`

- `score` is the name; `10` is the value.
- Later, evaluating `score` retrieves its current value.
- The name and the value are related, but they are not the same thing.

Assignment Evaluates First and Associates Second

Statement

```
score = 7 + 3
```

1. Evaluate the right-hand expression: `7 + 3` $\longrightarrow$ 10.
2. Associate the name `score` with the resulting value 10.

Image Placeholder

`assignment-evaluate-then-bind.png`

Key idea

Assignment Is Not Algebraic Equality

In algebra

$x = x + 1$ would assert that two quantities are equal, which is impossible for ordinary numbers.

In Python

`x = x + 1` is an instruction:

1. retrieve the current value of `x`;
2. add `1`;
3. associate `x` with the new result.

Key idea

Read = as "assign" or "associate," not "is mathematically equal to."

Program State Is a Snapshot of Current Associations

Definition

A program's **state** is the collection of variable–value associations that currently exist.

Statements executed

```
score = 10
bonus = 5
```

Name	Current value
score	10
bonus	5

Key idea

Executing an assignment can change the program's state; merely evaluating an expression does not create a new name.

Reassignment Replaces a Name's Current Association

Sequence

```
score = 10
score = 25
```

- After line 1: `score` $\longrightarrow$ 10
- After line 2: `score` $\longrightarrow$ 25
- The name has one current association in this simple model.

Image Placeholder

`reassignment-moves-binding.png`

Part 2: Names, Objects, and Memory

Python Connects Names to Objects

- An assignment **binds a name** to an object.
- The name is recorded in a **namespace**.
- The object carries a value, a type, and an identity.
- Evaluating the name asks its namespace which object is currently associated with that name.

Definition

A **namespace** is a mapping from names to objects.

Image Placeholder

`namespace-name-object-mapping.png`

Where Is the Variable Name Stored?

- At the top level of a Colab cell or Python file, assignment records a name in the module's global namespace.
- Python exposes a module namespace as a dictionary-like mapping.
- Conceptually, an entry looks like:

Namespace entry

`"score"` $\longrightarrow$ object representing 10

Example

The spelling `score` is stored separately from the numerical information in the integer object.

Image Placeholder

`variable-name-storage-namespace.png`

Every Python Object Has Identity, Type, and Value

Identity Distinguishes this object from every other object during its lifetime.

Type Determines which operations the object supports and helps determine how its value is represented.

Value The information represented by the object, such as `10` or `"hello"`.

Key idea

The type belongs to the object. A name can later be associated with an object of another type.

Image Placeholder

`object-identity-type-value.png`

`id()` Observes the Current Object's Identity

Colab experiment

```
score = 10
id(score)
```

- `id(score)` first evaluates `score` to retrieve its object.
- It then returns an integer that identifies that object during its lifetime.
- The particular integer is not meaningful to us and can differ across runs.
- It is the identity of the **object**, not an "address of the variable name."

Image Placeholder

`id-current-object-cpython-address.png`

Is an Object's Identity Its Memory Address?

Python's language-level promise

`id()` returns an integer that is unique and constant for the object during its lifetime.

CPython's implementation

In CPython, that integer is the object's memory address. Other Python implementations need not use an address.

- Memory management is automatic; programmers ordinarily do not choose these addresses.
- After an object no longer exists, a later object may reuse an identity value.
- Use `id()` to test the mental model, not to build arithmetic or program logic around the number.

Key idea

"Identity" is the portable idea; "memory address" is a CPython-specific implementation detail.

Assignment Changes the Namespace Entry

Before reassignment

`score = 10`

namespace: `score` $\longrightarrow$ integer object 10

After reassignment

`score = 25`

namespace: `score` $\longrightarrow$ integer object 25

Key idea

Reassignment does not edit the integer 10. It changes which object the name `score` denotes.

In class

Predict whether `id(score)` must remain the same after `score = 25`; then test the prediction in Colab.

String Reassignment Creates a New Result

Sequence

```
message = "hello"
message = message + " world"
```

1. Retrieve the current string `"hello"`.
2. Concatenate it with `" world"`.
3. Produce a new string `"hello world"`.
4. Rebind `message` to the new result.

Image Placeholder

`string-reassignment-new-object.png`

Why Are Strings Immutable?

Definition

A string is **immutable**: after a string object is created, its textual value cannot be altered.

- Text often represents identifiers, usernames, passwords, filenames, and dictionary or database keys.
- Stable string values are easier to share and reason about safely.
- Operations that appear to modify text instead produce another string value.

Example

`message += " world"` still produces a new string and rebinds `message` for the simple string case.

Image Placeholder

`string-immutability-stable-identifiers.png`

Part 3: Special Values and Dynamic Typing

`None` Represents the Intentional Absence of a Value

Example

```
result = None
```

- `None` is a literal value, not an unassigned name.
- Its type is `NoneType`.
- Python provides exactly one `None` object, so it is a singleton.
- A variable associated with `None` *has* a value: the value representing absence.

Example

`type(None)` $\longrightarrow$ `NoneType`

Image Placeholder

`none-singleton-object.png`

The “Billion-Dollar Mistake” Is a Warning About Absence

- Computer scientist Tony Hoare introduced a null reference into ALGOL W’s type system in 1965.
- In a 2009 talk, he called that decision his “billion-dollar mistake,” citing errors, vulnerabilities, and crashes caused by unchecked null references.
- Python’s `None` is not identical to every language’s null mechanism, but it carries the same general lesson: absence must be represented deliberately and handled explicitly.

Key idea

A value meaning “nothing here” is still part of program state and must be reasoned about.

Python Is Dynamically Typed

The same name can be rebound

```
item = 42                integer object
item = "hello"             string object
item = 4.0          floating-point object
```

- Each object has a definite type.
- The name `item` does not have one permanent type.
- Its current value and type depend on the most recent executed assignment.

Image Placeholder

`dynamic-rebinding-object-types.png`

Dynamic Typing Has Benefits and Costs

Benefits

- less type declaration syntax;
- rapid experimentation;
- flexible reuse of code.

Costs

- some type mismatches appear only when code runs;
- a name's intended kind of value may be less obvious;
- changes in state require careful reading.

Example

`item = 42` followed by `item = "hello"` is legal, but a reader must understand whether that change is intentional.

Type Annotations Communicate Intended Types

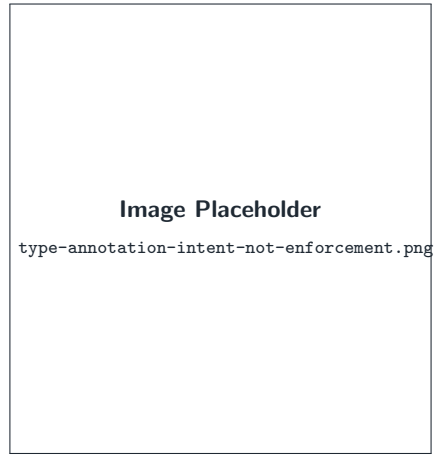
Annotated assignment

```
score: int = 42
```

- `: int` records that `score` is intended to refer to an integer.
- Editors, documentation tools, and type checkers can use the annotation.
- Standard Python execution ordinarily does not enforce the annotation by itself.

Example

An annotation supports communication and analysis; assignment still determines the current runtime association.



Part 4: Updating State

Updating a Variable Uses Its Current Value

Statement

```
score = score + 5
```

1. Look up the current object associated with `score`.
2. Evaluate the right-hand expression `score + 5`.
3. Associate `score` with the resulting object.

Key idea

Every use of `score` on the right occurs before the assignment changes `score` on the left.

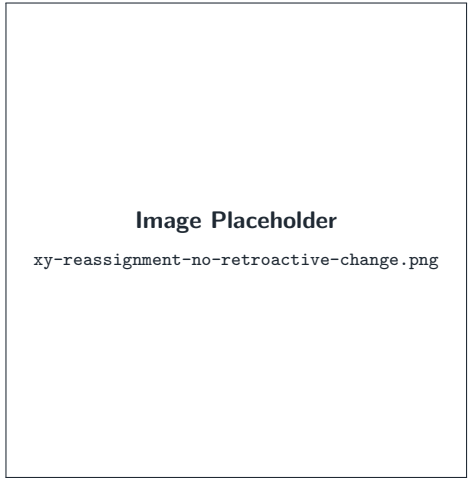
A Variable Retains a Result, Not the Formula

Sequence

```
x = 4
y = x + 2
x = 10
```

In class

What are the final values of `x` and `y`? Does the final assignment to `x` retroactively change `y`?



Key idea

Accumulation Repeatedly Updates One Association

Sequence

```
total = 0
total = total + 5
total = total + 8
```

After line	`total`
1	0
2	5
3	13

Key idea

Each update uses the current state to calculate the next state.

A Name Must Be Assigned Before It Is Evaluated

No current association

```
points + 5
```

- Python must retrieve `points` before it can perform addition.
- If no accessible binding for `points` exists, evaluation cannot continue.
- Python reports a `NameError`.

Example

Correct the state first: execute `points = 0`, then evaluate or update `points`.

Augmented Assignment Is a Compact Update

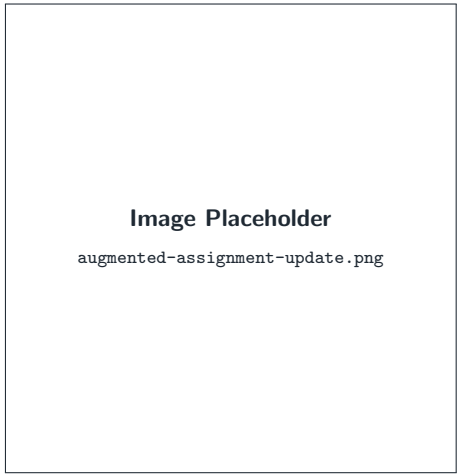
Ordinary reassignment

```
total = total + 5
```

Augmented assignment

```
total += 5
```

- Retrieve the current `total`.
- Add `5`.
- Associate `total` with the result.



Example

Common Augmented Assignment Operators

Operator	Update	Operator	Update
`+=`	add, then reassign	`-=`	subtract, then reassign
`*=`	multiply, then reassign	`/=`	divide, then reassign
`//=`	floor-divide, then reassign	`%=`	take modulus, then reassign
`**=`	exponentiate, then reassign		

Example

If `count` currently refers to 8, then `count *= 3` associates `count` with 24.

Key idea

Introduce the compact form only after you can explain the ordinary reassignment it represents.

Part 5: Readable State and Deliberate Practice

Comments Explain Code Without Changing State

End-of-line comments

```
score = 10 # starting score
score += 5 # add the bonus
```

- A comment begins with # and continues to the end of that line.
- Python ignores the comment when executing the statement.
- A useful comment explains purpose or reasoning, not merely the visible syntax.

Image Placeholder

`comments-versus-state-changes.png`

Technical distinction

Triple-quoted text creates a string literal. It is sometimes used for documentation, but it is

Variable Naming: Valid, Descriptive, and Conventional

Python's basic identifier rules

- use letters, digits, and underscores;
- do not begin with a digit;
- do not include spaces or hyphens;
- capitalization matters;
- do not use a reserved keyword.

Course convention

Use lowercase words separated by underscores: `course_units`, `total_cost`, `quiz_score`.

Reserved keywords

Python reserves words such as `if`, `else`, `for`, `while`, `class`, `def`, `return`, `import`, `True`, `False`, and `None`. They cannot be variable names.

Example

Prefer `exam_score = 92`, not `x = 92`, when the value's role matters.

Keywords and Built-in Names Are Different

Reserved keywords

`if = 3` is invalid syntax because `if` is part of Python's grammar.

- Keywords cannot be used as variable names.
- Python publishes the current keyword list through the `keyword` module.

Built-in names

`print = 3` is legal, but it hides the built-in name `print` in the current namespace.

- Avoid names such as `print`, `type`, `id`, `int`, and `str`.
- Legal does not always mean readable or safe.

Key idea

A keyword is forbidden by the grammar; a built-in name is merely easy to shadow accidentally.

Colab Lab: Observe Rebinding with `id()`

Run one line at a time

```
message = "hello"
id(message)
message += " world"
id(message)
```

In class

Before running the second `id()` call, predict whether the object identity will be the same. Explain the observation using “evaluate,” “new string,” “namespace,” and “rebind.”

Image Placeholder

`colab-id-string-reassignment.png`

Interpretation rule

Misconceptions to Correct Explicitly

- `x = x + 1` is valid because assignment is an action, not an algebraic claim.
- The entire right-hand side is evaluated before the left-hand name is updated.
- A variable retains the resulting object, not the expression that produced it.
- Reassigning `x` does not retroactively update `y`.
- A name must be assigned before Python can evaluate it.
- `id(x)` identifies the current object, not the storage location of the written name `x`.
- A name can be rebound to an object of another type; the objects themselves still have definite types.

Exit Ticket: Explain the Final State

Statements

```
level = 2
next_level = level + 1
level = 10
level += 2
```

In class

Report the final values of `level` and `next_level`. Then explain:

1. what each assignment evaluates;
2. which namespace association changes;
3. why `next_level` does not change later.

Key idea

Outcome: you can predict how a sequence of assignments changes program state and explain that state using names, objects, and values.

Image Placeholder Index

- `expression-versus-stateful-program.png`
- `assignment-evaluate-then-bind.png`
- `reassignment-moves-binding.png`
- `namespace-name-object-mapping.png`
- `variable-name-storage-namespace.png`
- `object-identity-type-value.png`
- `id-current-object-cpython-address.png`
- `string-reassignment-new-object.png`
- `string-immutability-stable-identifiers.png`
- `none-singleton-object.png`
- `tony-hoare-null-reference.png`
- `dynamic-rebinding-object-types.png`
- `type-annotation-intent-not-enforcement.png`
- `xy-reassignment-no-retroactive-change.png`
- `augmented-assignment-update.png`
- `comments-versus-state-changes.png`
- `colab-id-string-reassignment.png`

Example

Save each image under `../assets/lecture-05/`. The labeled placeholder will be replaced automatically when its filename matches.